

Spring Boot — Lecture 1

Introduction to Spring Framework

1. About This Series

This series covers Spring Framework and Spring Boot from scratch — beginner friendly, but goes deep into internals so you actually understand how things work, not just the syntax.

- Covers Spring Core, Spring MVC, Spring Boot — internals, not just syntax.
- Practical industry knowledge shared throughout — what's actually used on the job.
- Interview preparation — understanding the architecture helps crack any Spring interview.
- Notes + practice questions provided after every lecture on the Git channel.

Why not just learn Spring Boot syntax alone? Because Spring Boot is built on Spring Core, Spring MVC, Spring Data, etc. Without knowing the internals you cannot debug, scale, or answer interview questions properly.

2. Client-Server Architecture

When you visit amazon.in, your browser sends a request to Amazon's server (a computer where their application is hosted). The server processes it and sends back a response which the browser displays. This is the **Client-Server Model**.

Entity	Role	Examples
Client	Sends the HTTP request	Browser, Mobile App, React Frontend, Postman, Another Microservice
Server	Receives request, processes it, sends response	Spring Boot App running on a machine or cloud

A server can also act as a client — e.g. in Microservices, Order Service calls Payment Service via HTTP. The Order Service is the client in that call.

What the server does when processing a request:

- Receive and parse the incoming HTTP request
- Authenticate the user (e.g. check login credentials)
- Perform business logic (fetch from DB, update data, delete, etc.)
- Build and return an HTTP response

3. HTTP — HyperText Transfer Protocol

Protocol = a rulebook / standard. HTTP defines *how* a client and server communicate — the exact format of requests, responses, and how data travels over the internet. Without a common protocol, client and server couldn't understand each other.

HTTP vs HTTPS:

Protocol	Encryption	Default Port	When used
HTTP	None — plain text, can be intercepted	80	Old websites, internal tools
HTTPS	End-to-end encrypted (S = Secured)	443	All modern websites — prevents eavesdropping

HTTP Methods:

Method	Purpose	Has Request Body?
GET	Retrieve / fetch data — read only	No

Method	Purpose	Has Request Body?
POST	Create / send new data to server	Yes
PUT	Update an entire existing resource	Yes
PATCH	Update a specific field of a resource	Yes
DELETE	Remove / delete a resource	No

4. HTTP Request Structure — 4 Parts

Part	Description	Example
Method	Type of operation being performed	GET / POST / PUT / DELETE / PATCH
URL / Path	Host + endpoint being targeted	www.coderarmyarmy.in/courses
Headers	Optional key-value metadata about the request	Accept: application/json Content-Type: application/json
Body	Data payload — only for POST, PUT, PATCH	{ "email": "user@gmail.com", "password": "abc123" }

GET request example (no body needed):

```
GET www.coderarmyarmy.in/courses Accept: application/json
```

POST request example — Login (has body):

```
POST www.coderarmyarmy.in/login Content-Type: application/json { "email": "user@gmail.com", "password": "secret123" }
```

Headers carry extra metadata (what format you accept, your auth token, etc.). Body carries the actual data payload — only present in POST / PUT / PATCH requests.

5. HTTP Response Structure — 3 Parts

Part	Description	Example
Status Code	Indicates whether the request succeeded or failed	200 OK, 201 Created, 404 Not Found, 500 Server Error
Headers	Metadata about the response	Content-Type: application/json
Body	Actual data returned by the server	{ "message": "Login successful" }

Common HTTP Status Codes:

Code	Meaning	When it happens
200	OK — Success	Request processed successfully
201	Created	A new resource was successfully created (e.g. user registered)
404	Not Found	The endpoint or resource doesn't exist on the server
500	Internal Server Error	Something crashed or went wrong on the server side
503	Service Unavailable	Server is overloaded or temporarily down

6. How Java Runs on a Server

Every API endpoint ultimately maps to a **Java method** inside a class on the server. When you hit /courses, a specific method runs. When you hit /login, a different method runs. Spring Boot makes this mapping easy.

Java execution flow (recap):

Step	What Happens
1	Write .java source file
2	Compile with javac → produces .class bytecode (platform independent)
3	JVM loads and executes the bytecode — runs inside a sandbox
4	For a web server: JVM stays alive 24/7, continuously listening for requests

Normal Java program vs Web Server:

Normal Java Program	Web Server (e.g. Spring Boot)
Start → Run instructions → Exit	Start → Wait for request → Process → Send response → Keep running (repeat forever)

A website is NOT a program that runs once and exits. It stays alive 24/7, continuously listening for incoming requests.

The mapping problem — what Core Java can't do alone:

Core Java (java.net package) can listen on a port using ServerSocket. But it receives raw bytes — the JVM has no idea what HTTP is. You'd have to do everything manually:

- Read the raw input stream of bytes
- Manually parse it into an HTTP request (extract method, URL, headers, body)
- Manually map the URL to the correct Java method (if/else chains)
- Manually build the HTTP response in the correct format
- Manually implement multi-threading to handle multiple users at once

This is massive boilerplate for every project. That's the problem Servlet + Tomcat solved.

7. Servlet + Servlet Container (1997)

Introduced as Java EE. A **Servlet** is a Java class that runs inside a **Servlet Container** (also called a Web Server). The container handles all the boilerplate so you only write business logic.

Servlet Container	What It Does For You
Apache Tomcat (most popular)	Opens the port, reads raw bytes, converts to HTTP request, routes to correct Servlet, builds HTTP response, manages threads for concurrent users.
Jetty	Lightweight alternative to Tomcat — used in embedded scenarios.
Undertow	High-performance server used by WildFly / JBoss.

Tomcat stays alive 24/7, listens for requests, and calls the correct Servlet based on the URL. You write logic inside the Servlet — Tomcat handles everything else.

Request flow with Servlet:

Step	Who does it	What happens
1	Client	Sends HTTP request to server IP:port
2	Tomcat	Reads raw bytes → parses into HttpServletRequest object
3	Tomcat	Looks up which Servlet is mapped to this URL → calls it
4	Servlet	Your Java code runs — business logic, DB calls, etc.
5	Tomcat	Takes Servlet's output → builds HTTP response → sends to client

8. Why Spring Framework?

Servlet-based code worked, but at enterprise scale the application became **tightly coupled** — objects deeply interconnected, making it hard to scale, test, and maintain. Spring introduced a philosophy to fix this.

Spring's core solution:

- Dependency Injection (DI) — objects receive their dependencies from outside, not create them
- Inversion of Control (IoC) — the framework controls object creation and lifecycle, not your code
- Beans — Spring-managed objects that live in an IoC container

Result: loosely coupled applications that are easier to scale, test, and maintain.

9. The Spring Ecosystem

Spring is not just one framework — it's an entire ecosystem of modules, all built on a common core.

Layer	Module	Purpose
Foundation	Spring Core	DI, IoC, Beans — the base ideology that everything else is built on.
Web	Spring MVC	Build REST APIs and web apps. Uses Servlets internally. Core skill to learn.
Data	Spring Data / JPA	Connect to databases. Built on Hibernate → JPA → JDBC stack.
Security	Spring Security	Authentication, Authorization, Login, JWT, OAuth2.
Cross-cutting	Spring AOP	Aspect Oriented Programming — logging, transactions, security across layers.
Automation	Spring Boot	Auto-configures all the above. Speeds up project setup. Not a replacement — a wrapper.
Modern	Spring AI	Build AI-powered applications using Spring conventions.

Spring Boot ≠ replacement for Spring Framework. It's an opinionated automation layer on top of Spring MVC, Spring Data, Spring Security etc. All the underlying modules are still there — Boot just auto-configures them.

10. Spring Boot — What It Actually Is

Spring Boot is an **opinionated framework** — it has its own pre-defined configurations and assumes sensible defaults so you can start writing business logic immediately without setting up boilerplate.

Without Spring Boot (Spring MVC alone)	With Spring Boot
Must manually configure DispatcherServlet	Auto-configured on startup
Must configure embedded server (Tomcat) separately	Tomcat embedded and started automatically
Must define beans and component scanning manually	Auto-detected via @SpringBootApplication
Must configure data sources, JPA, etc. manually	Auto-configured from application.properties
Slower project startup — lots of boilerplate config	Start writing business logic in minutes

You can override any of Spring Boot's defaults when needed. But to do that correctly, you must understand the internals — which is why we learn Spring Core and Spring MVC first in this series.

11. Database Layer Evolution

Understanding the full data access stack helps you debug issues at any layer.

Technology	What you write	Notes
JDBC	Raw SQL queries in Java code	Manual, verbose — full control but lots of boilerplate.
JPA (spec)	Method calls — no SQL written	Just a specification / rulebook, not an implementation itself.

Technology	What you write	Notes
Hibernate	Implements JPA, generates SQL internally	Most popular JPA implementation — uses JDBC underneath.
Spring Data JPA	Just interface methods — zero boilerplate	Built on top of Hibernate → JPA → JDBC. Highest abstraction.

Full chain: Spring Data → Hibernate → JPA → JDBC → Database. You need to understand the full chain to debug data access issues properly.

12. Complete Spring Boot Application Architecture

Layer	Technology	Responsibility
Client	Browser / Mobile App / Postman	Sends HTTP request to the server
Web Layer	Spring MVC (Tomcat embedded)	Routes request to correct controller method via @GetMapping etc.
Controller	Your @RestController classes	Receives request, calls service layer, returns response
Service	Your @Service classes	Business logic — calculations, validations, orchestration
Security	Spring Security	Authenticate and authorize the request before it reaches controller
Data Layer	Spring Data → Hibernate → JDBC	Reads from / writes to the database
Database	MySQL / Oracle / PostgreSQL	Stores and retrieves persistent data

Monolithic vs Microservices:

Architecture	Description	When to use
Monolithic	One single Spring Boot app handles all endpoints and features	Small teams, simple apps, early-stage projects
Microservices	Multiple small Spring Boot apps (Order Service, Payment Service, User Service) communicating via HTTP	Large teams, complex domains, independent scaling needed

Knowing the complete architecture is what makes you a Java Full Stack Developer. This entire series builds toward mastering every layer shown above.

Next Lecture: Building First Spring Boot Hello World App + Spring Initializr